

Backpropagation by Hand

From the loss to the gradients, without a computation graph

Pierluigi Covone

June 12, 2026

Contents

1	Introduction	1
2	Forward pass	2
3	Backpropagation	3
3.1	Derivative of the loss with respect to P	3
3.2	The Jacobian of the softmax	4
3.3	Derivative of the loss with respect to the logits	4
3.4	Derivative of the loss with respect to the weights	5

1 Introduction

The aim of this document is to explain the mathematics the backward pass implemented in `neural.py` comes from; read in isolation, the code looks almost unrelated to the theoretical formulas (due to the fact that, when translating into code, things must “fit” machine requirements). None of those lines was written by guessing or by copying a known recipe. Each one was reached the long way: by working out on paper what the gradient actually *is*, and only then asking what the code has to do. This document records that paper work, so that the distance between the textbook formulas and the “targeted” code can be walked in both directions.

The backward pass follows the steps of the forward pass in reverse, breaking the pipeline into sub-parts (which we call *links*), each with its own dedicated gradient computation. There is no computation graph underneath, no engine tracking operations and replaying them backwards — only the chain rule, applied on paper, one link at a time.

The intent is educational: modern frameworks ship these gradients ready to use, computed behind a single function call. The goal here is not to reproduce what already exists, but to deeply understand how it works — to see *why* the gradient (here, of this specific network) takes the shape it takes, rather than to take that shape on trust.

2 Forward pass

The data. The training set consists of two integer arrays of length N , where N is the number of bigrams observed in the dataset¹:

- xs : the input characters, mapped to their indices in the vocabulary;
- ys : the targets — y_k is the target character of example k , mapped to its index.

The network cannot consume raw indices²: xs is first transformed into a one-hot encoded matrix $X_{\text{enc}} \in \mathbb{R}^{N \times V}$, one row per example, where V is the vocabulary size (including the boundary token). Row k is all zeros except for a 1 in column x_k .

Why W is $V \times V$. The network has a single linear layer, so that layer is at the same time the first and the last one — and each of the two roles fixes one of its dimensions.

- As the *first* layer, its input dimension is dictated by the *encoding*: each neuron receives one input per column of the one-hot representation, hence V inputs.
- As the *last* layer, its output dimension is dictated by the *task*: we want a probability distribution over the V characters of the vocabulary, hence V outputs.

Therefore $W \in \mathbb{R}^{V \times V}$.

Overview. The whole computation, from input to loss, is a short pipeline:

$$X_{\text{enc}} \xrightarrow{\cdot W} Z \xrightarrow{\text{softmax}} P \xrightarrow{\text{NLL}} \mathcal{L}, \quad \text{where:}$$

- $Z = X_{\text{enc}} W \in \mathbb{R}^{N \times V}$ is the matrix of *logits*;
- $P = \text{softmax}(Z) \in \mathbb{R}^{N \times V}$ is the matrix of probabilities:
row k is the predicted distribution over the next character for example k ;
- $\mathcal{L} \in \mathbb{R}$ is the loss, a single scalar measuring how well P agrees with the targets ys .

Softmax. Applied row-wise, it turns each row of unconstrained logits into a probability distribution:

$$p_{k,j} = \frac{e^{z_{k,j}}}{\sum_{m=0}^{V-1} e^{z_{k,m}}}, \quad \text{where:}$$

- $k \in [0, N-1]$ indexes the rows (one per example);
- $j \in [0, V-1]$ indexes the columns (one per character of the vocabulary);
- $z_{k,j}$ is the corresponding logit, and the sum over m runs across the whole row k .

In the implementation, the maximum of each row is subtracted from its logits before exponentiating: e^z explodes quickly for large z , and the subtraction keeps `np.exp()` from overflowing. The result is unchanged — softmax is invariant under shifting a whole row by a constant.

Negative log-likelihood. The loss is the mean negative log-likelihood:

$$\mathcal{L} = -\frac{1}{N} \sum_{k=0}^{N-1} \log p_{k,y_k}, \quad \text{where:}$$

- p_{k,y_k} is the probability that row k assigns to its own target y_k : each example contributes a single entry of P , the one in the target's column.

¹The dataset used is: *names.txt*.

²Indices are not good inputs because the net would treat them as quantitative, but the information is qualitative.

3 Backpropagation

The forward pass reads left to right; backpropagation walks the same pipeline right to left, asking at each link: *how does the loss change if this quantity moves?* The steps, in the order we will derive them, are:

1. $\partial\mathcal{L}/\partial P$ — the derivative of the loss with respect to the probabilities (Section 3.1);
2. $\partial P/\partial Z$ — the Jacobian of the softmax (Section 3.2);
3. $\partial\mathcal{L}/\partial Z$ — the derivative with respect to the logits, obtained by composing the previous two via the chain rule (Section 3.3);
4. $\partial\mathcal{L}/\partial W$ — the derivative with respect to the weights, the last link of the chain (Section 3.4).

A note on what ends up in the code: on paper, each step feeds the next; the chain rule composes them all. In the implementation, however, not every intermediate piece is needed as a stepping stone. As we will see in Section 3.3, the composition collapses into an expression so compact that the intermediate results it was built from never need to be materialized.

Method. Every link of the chain is attacked the same way: first we shrink the sub-problem to its simplest instance, then we extend the result to the real one. In our case the simplification is always the same — work on a single row (a single example), since examples contribute independent terms to the loss — and so is the extension: replicate the result across the N rows and reintroduce the $1/N$ factor of the mean.

Verification. Every analytic gradient derived is backed by a test that computes its numerical counterpart from the very definition of derivative: each entry is perturbed by $\pm\varepsilon$ and the change in the loss is measured via central differences,

$$\frac{\partial\mathcal{L}}{\partial x} \approx \frac{\mathcal{L}(x + \varepsilon) - \mathcal{L}(x - \varepsilon)}{2\varepsilon}.$$

3.1 Derivative of the loss with respect to P

Reasoning on a single example — that is, on a single row p of P — the contribution of that example to the loss is

$$\ell = -\log p_y, \quad \text{where:}$$

- y is the index of the target character for this example;
- p_y is the probability the row assigns to that target.

We now ask how ℓ changes when each entry p_j of the row moves. Looking at the formula, ℓ depends on *one single* entry: p_y . Every other p_j with $j \neq y$ simply does not appear. Hence:

$$\frac{\partial\ell}{\partial p_j} = \begin{cases} -\frac{1}{p_y} & \text{if } j = y, \\ 0 & \text{if } j \neq y. \end{cases}$$

The gradient $\partial\ell/\partial p$ is therefore an extremely *sparse* vector: zero everywhere except at the target position.

Extending to N examples: the loss is the mean of N terms, and each term depends only on its own row of P . Differentiating with respect to the entry $p_{k,j}$, every term of the mean vanishes except the k -th one, which carries its $1/N$ factor along. The full gradient $\partial\mathcal{L}/\partial P$ is therefore a matrix with the same shape as P , zero everywhere except at one entry per row:

$$\frac{\partial\mathcal{L}}{\partial p_{k,j}} = \begin{cases} -\frac{1}{N p_{k,y_k}} & \text{if } j = y_k, \\ 0 & \text{otherwise.} \end{cases}$$

In the code, we simply allocate a matrix of zeros with the shape of P , then fill the target cell of each row with $-1/(N p_{k, y_k})$.

3.2 The Jacobian of the softmax

The next link in the chain is the softmax. Following our method, we fix a single row: one row of logits and the probability row that the softmax produces from it. We now need $\partial p_j / \partial z_i$: how the j -th probability of the fixed row changes when we move the i -th logit of the *same row* — same row, but not necessarily the same column: i and j are free to differ, and the two cases below depend precisely on whether they coincide or not. Recall the formula (the k row-index is dropped, having fixed the row):

$$p_j = \frac{e^{z_j}}{S}, \quad S = \sum_{m=0}^{V-1} e^{z_m}.$$

Look at the formula of p_j : it is a fraction, and the logit z_i can appear in the numerator, in the denominator, or in both.

The denominator first. S sums the exponentials of *all* the logits of the fixed row, $z_0, \dots, z_{V-1}$ (each one produced by the linear forward $Z = X_{\text{enc}} W$): the index m in the sum runs across that row. Whatever i we pick, z_i is one of them: the denominator *always* depends on z_i . This is why every logit influences every probability of its row — a sharp contrast with the previous step, where each entry of the loss touched a single probability.

The numerator instead contains a single exponential, e^{z_j} : it depends on z_i only when $i = j$. This is the discriminating factor, and it splits the computation in two cases.

Case $i = j$. The logit appears both in the numerator and in the denominator. Applying the quotient rule to $p_i = e^{z_i} / S$, with $\partial S / \partial z_i = e^{z_i}$:

$$\frac{\partial p_i}{\partial z_i} = \frac{e^{z_i} S - e^{z_i} e^{z_i}}{S^2} = \frac{e^{z_i}}{S} - \left(\frac{e^{z_i}}{S} \right)^2 = p_i - p_i^2 = p_i (1 - p_i).$$

Case $i \neq j$. Now e^{z_j} is a constant with respect to z_i ; the only dependence is through the denominator S . Since $\frac{\partial}{\partial z_i} \left(\frac{1}{S} \right) = -\frac{e^{z_i}}{S^2}$ (power rule $S^{-1} \rightarrow -S^{-2}$, times $\partial S / \partial z_i = e^{z_i}$):

$$\frac{\partial p_j}{\partial z_i} = e^{z_j} \left(-\frac{e^{z_i}}{S^2} \right) = -\frac{e^{z_j}}{S} \cdot \frac{e^{z_i}}{S} = -p_j p_i.$$

Note that both results are expressed *purely in terms of p* : the exponentials have disappeared. This is the key to the simplification in the next section. Unlike the other gradients in this chain, the Jacobian has no dedicated test: it is an *intermediate step*. It will be validated indirectly in the next section, where its composition with $\partial \ell / \partial p$ is checked numerically.

3.3 Derivative of the loss with respect to the logits

We now hold both pieces — $\partial \ell / \partial p$ from Section 3.1 and the Jacobian from Section 3.2 — and the chain rule composes them. Still on the fixed row: we just saw that the logit z_i influences *every* probability of its row, and the loss reads those probabilities; so the loss can reach z_i through V distinct paths, one per probability, and each path contributes a term:

$$\frac{\partial \ell}{\partial z_i} = \sum_j \frac{\partial \ell}{\partial p_j} \frac{\partial p_j}{\partial z_i}, \quad \text{where:}$$

- $j \in [0, V - 1]$ iterates over the V paths — which are nothing but the entries (the columns) of the fixed probability row: the loss reaches z_i once through each of them.

But Section 3.1 also tells us that $\partial\ell/\partial p_j$ is zero everywhere except at $j = y$: of the V paths, only the one through the row’s target probability survives,

$$\frac{\partial\ell}{\partial z_i} = \frac{\partial\ell}{\partial p_y} \frac{\partial p_y}{\partial z_i} = -\frac{1}{p_y} \frac{\partial p_y}{\partial z_i}.$$

One question remains: is z_i the target’s logit or not? The Jacobian answers it: there we computed it for a generic pair (i, j) ; here the probability is fixed at $j = y$, so we read its two cases with y in place of j .

Substituting into $\partial\ell/\partial z_i = -\frac{1}{p_y} \cdot \partial p_y/\partial z_i$, the $1/p_y$ cancels in both branches, and we obtain:

$$\frac{\partial\ell}{\partial z_i} = \begin{cases} -\frac{1}{p_y} \cdot p_y (1 - p_y) = p_y - 1 & \text{if } i = y, \\ -\frac{1}{p_y} \cdot (-p_y p_i) = p_i & \text{if } i \neq y. \end{cases}$$

These are the derivatives with respect to a *single* logit z_i . The gradient with respect to the whole row z is the vector that collects them, for $i = 0, \dots, V - 1$. Look at what that vector contains: position by position, it holds the probability p_i — except at the target position, where a 1 is subtracted from it. But “all zeros, with a 1 at the target position” is exactly $\text{onehot}(y)$. The whole vector is therefore p with $\text{onehot}(y)$ subtracted:

$$\boxed{\frac{\partial\ell}{\partial z} = p - \text{onehot}(y).}$$

Extending to the N rows, which behave identically, and reintroducing the $1/N$ of the mean:

$$\boxed{\frac{\partial\mathcal{L}}{\partial Z} = \frac{1}{N} (P - \text{onehot}(ys)).}$$

What ends up in the code. This is the collapse anticipated at the beginning of Section 3: the final expression contains no trace of $\partial\mathcal{L}/\partial P$, nor of the Jacobian — they cancelled each other into a subtraction. On paper, both were necessary to *prove* the result; in the implementation, neither is needed to *compute* it. In the code, the function `d_loss_d_logits` simply builds the one-hot matrix of the targets, subtracts it from P , and divides by N .

Interpretation. The result $p - \text{onehot}(y)$ reads naturally: the gradient on the logits is “*what the model predicts*” minus “*what was correct*”. If the model were perfect (p equal to the target’s one-hot), the gradient would be *zero*: nothing to correct. The gradient, in other words, *is* the prediction error: it measures how much, and in which direction, each logit would need to move for the loss to decrease. The logits themselves are never touched directly — only W is updated — but this error signal is exactly what travels backwards to reach the weights.

3.4 Derivative of the loss with respect to the weights

The last link: propagating the gradient from the logits back to the weights, through the linear forward $Z = X_{\text{enc}} W$. From the previous section we hold $\partial\mathcal{L}/\partial Z$ in our hands; what we need now is $\partial\mathcal{L}/\partial W$ — how the loss changes when each weight moves. Note that the target has changed nature: so far we differentiated with respect to intermediate *results* (P, Z); this time the variable is a *parameter* of the network: the matrix the training loop will actually update.

Following our method, we fix a single example: one row x of X_{enc} and the row z of Z (the logits) it produces. Within the row, fix an arbitrary position i : the linear forward computes that logit as

$$z_i = \sum_j x_j w_{j,i}, \quad \text{where:}$$

- $i \in [0, V - 1]$ is the fixed position: z_i sits at column i of the logits row, and is computed using only the weights of *column* i of W ;
- $j \in [0, V - 1]$ is the summation index: at each step it picks the input x_j and the matching weight $w_{j,i}$ from the fixed column i .

One subtlety before differentiating. In the previous sections the variable of interest carried one index; a weight carries *two*. To avoid any collision with the indices i, j already used inside the sum, we give the weight fresh letters: pick a generic $w_{a,b}$, sitting at row a , column b of W .

How does the logit z_i react when $w_{a,b}$ moves? Look at the sum: the only weights appearing in z_i are $w_{0,i}, \dots, w_{V-1,i}$ — those of *column* i . So everything depends on whether our weight belongs to that column or not:

- if $b \neq i$, the weight $w_{a,b}$ lives in a different column: it does not appear in the sum at all, and the derivative is 0;
- if $b = i$, the weight belongs to the right column — but the sum runs down its rows, one term per row j , and $w_{a,b}$ occupies exactly one of them: row a . It therefore appears in the single term $j = a$, namely $x_a w_{a,i}$; every other term is constant with respect to it, and the derivative is x_a .

$$\frac{\partial z_i}{\partial w_{a,b}} = \begin{cases} x_a & \text{if } i = b, \\ 0 & \text{if } i \neq b. \end{cases}$$

Now the chain rule. The weight $w_{a,b}$ does not touch the loss directly: its influence travels through the logits, which become probabilities, which enter the loss. Every logit of the fixed row is, in principle, a possible path: to account for the total effect of moving $w_{a,b}$, we must sum the contribution of each route — “how much the loss moves when z_i moves” times “how much z_i moves when $w_{a,b}$ moves”:

$$\frac{\partial \ell}{\partial w_{a,b}} = \sum_i \frac{\partial \ell}{\partial z_i} \frac{\partial z_i}{\partial w_{a,b}}, \quad \text{where:}$$

- $i \in [0, V - 1]$ iterates over the V paths, one per logit of the fixed row;

But the derivative just computed, $\partial z_i / \partial w_{a,b}$, is zero for every i except $i = b$: of the V paths, only the one through the logit of the weight’s own column survives. Keeping only the term $i = b$, where the derivative is worth x_a , we obtain:

$$\frac{\partial \ell}{\partial w_{a,b}} = \frac{\partial \ell}{\partial z_b} \cdot x_a.$$

Both coordinates of the weight are visible in the result: the column b selected the surviving path, and shows up in which entry of dlogits is used; the row a selected the surviving term inside that logit’s sum, and shows up as the input component x_a .

Note what this formula does *not* require: reopening the target/non-target cases. That work is already packaged inside $\partial \ell / \partial z_b$. This is how the chain extends, here and in any deeper network: each new link consumes the previous one as a closed box.

From one entry to the whole matrix. Since a and b are arbitrary, the formula holds for *every* entry of dW : entry (a, b) is “component a of x ” times “component b of $\partial\ell/\partial z$ ”. A matrix built this way is, by definition, the *outer product* of the two vectors:

$$\frac{\partial\ell}{\partial W} = x^\top \frac{\partial\ell}{\partial z}, \quad (V \times 1)(1 \times V) = V \times V.$$

From one example to N . So far the example was fixed. But W does not belong to one example: the *same* matrix is traversed by all N of them in the forward pass, so the loss — a mean over the examples — depends on each weight through every example. When we differentiate the loss with respect to W , then, each example produces its own outer product; the total gradient is the sum of all of them:

$$\frac{\partial\mathcal{L}}{\partial W} = \sum_{k=0}^{N-1} x_k^\top g_k, \quad \text{where:}$$

- x_k is the k -th row of X_{enc} (the k -th input);
 - g_k is the k -th row of dlogits, the gradient of the loss with respect to the k -th row of logits.
- No extra $1/N$ is needed: the mean factor is already inside dlogits.

A sum of outer products is exactly what a matrix product computes:

$$\frac{\partial\mathcal{L}}{\partial W} = X_{\text{enc}}^\top \cdot \text{dlogits}.$$

The shapes confirm it: $(V \times N)(N \times V) = V \times V$, the shape of W . In the code, the whole computation — outer products and sum over the examples — is a single matrix multiplication: `xenc.T @ dlogits`.

Interpretation. Recall that the formula for a single example was $\partial\ell/\partial w_{a,b} = x_a \cdot \partial\ell/\partial z_b$, and that x is one-hot: every component is 0 except the one at the input character’s index. So, for almost every row a of W , the factor x_a is zero and the whole row of gradients vanishes. Only one row survives: the row of the character the example saw as input. In other words, each example adjusts only the weights it actually used — in the forward pass, multiplying the one-hot x by W had selected exactly that row and ignored every other. What a row of W does not touch in the forward, it does not answer for in the backward.